

Lógica Computacional

Práctica 11: Introducción a Rocq

Erick Daniel Arroyo Martínez

Manuel Soto Romero

Semestre 2026-2
Facultad de Ciencias UNAM

Objetivos

1. Familiarizarse con el entorno Rocq y su modelo de interacción basado en metas.
2. Comprender la noción de *tipo*, *expresión* y *evaluación* en un sistema funcional puro.
3. Definir funciones mediante *Definition* y *Fixpoint*; explorar la recursión estructural.
4. Introducir tipos inductivos y *pattern matching*.
5. Construir demostraciones con tácticas básicas: *intros*, *simpl*, *reflexivity*, *rewrite* y *destruct*.
6. Comprender la diferencia entre *Qed*, *Admitted* y *Abort*.

Instrucciones

1. No es necesario instalar Rocq localmente.
2. **Toda** la práctica se realizará utilizando el entorno: <https://jscoq.github.io/scratchpad.html>
3. **No** se permite el uso de automatización avanzada ni tácticas no vistas en laboratorio.
4. En esta práctica únicamente se utilizarán las tácticas:

```
intro  intros  simpl  reflexivity  rewrite  destruct
```

5. Las demostraciones deben construirse **manualmente**.
6. Cada ejercicio indica claramente si debe terminar con *Qed* o si basta con *Admitted*.

Marco teórico

¿Qué es Rocq? Rocq (anteriormente llamado *Coq*) es un asistente de pruebas basado en el *Cálculo de Construcciones Inductivas*. La computación en Rocq no se entiende como una secuencia de instrucciones que modifican un estado, sino como *reducción de expresiones*: un programa es un término, y ejecutarlo significa simplificarlo paso a paso hasta alcanzar su forma [1].

La perspectiva de *Curry–Howard* identifica proposiciones con tipos y demostraciones con programas. Por eso, en Rocq hay un único lenguaje para escribir funciones y demostrar teoremas.

Tipos fundamentales. Todo término tiene un tipo. El comando `Check` permite [1]:

```

Check nat.      (* nat : Set *)
Check bool.     (* bool : Set *)
Check True.     (* True : Prop *)
Check 3.        (* 3 : nat *)

```

Los tipos más usados en esta práctica son:

- `nat`: números naturales (definidos inductivamente).
- `bool`: booleanos (`true` / `false`).
- `Prop`: universo de proposiciones lógicas.

Definiciones y evaluación.

```

Definition doble (n : nat) : nat := n + n.

Compute doble 5.    (* = 10 : nat *)

```

Tipos inductivos. Los naturales están predefinidos, pero su definición conceptual es:

```

Inductive nat : Type :=
| 0 : nat
| S : nat -> nat.

```

Así, $0 = 0$, $1 = S0$, $2 = S(S0)$, etc. Cualquier tipo inductivo declara sus *constructores*; fuera de ellos, no existe ningún habitante del tipo [2].

Pattern matching. El análisis por casos sobre un dato inductivo se escribe con `match` :

```

Definition predecesor (n : nat) : nat :=
  match n with
  | 0   => 0
  | S k => k
  end.

```

Recursión estructural con Fixpoint. Las funciones recursivas se introducen con `Fixpoint`. Rocq verifica que cada llamada recursiva opera sobre un argumento *estructuralmente menor*, lo que garantiza terminación:

```

Fixpoint suma (n m : nat) : nat :=
  match n with
  | 0   => m
  | S k => S (suma k m)
  end.

```

Proposiciones y teoremas. Las proposiciones viven en `Prop`. Un teorema declara una proposición y obliga a demostrarla [1]:

```
Theorem nombre : proposicion.
Proof.
  (* tactics *)
Qed.
```

`Qed`, `Admitted` y `Abort`.

- `Qed` : cierra la prueba verificándola formalmente. Rocq rechaza un `Qed` si quedan metas abiertas.
- `Admitted` : acepta el teorema *sin prueba*, marcándolo internamente como un axioma no justificado. Útil para avanzar cuando una demostración aún no está lista, pero contamina la cadena de confianza.
- `Abort` : descarta la prueba en curso sin dejar el enunciado declarado. Sirve para cancelar un intento fallido.

Tácticas básicas. [1]

`intros` Mueve al contexto local las variables universalmente cuantificadas y las hipótesis de una implicación. `intro h` mueve una sola y la nombra `h`.

`simpl` Reduce expresiones computables: despliega definiciones, evalúa *pattern matching*, etc.

`reflexivity` Cierra una meta de la forma $t = t$ (o cualquier igualdad cuyos dos lados reduzcan al mismo término).

`rewrite H` Si $H : a = b$, sustituye `a` por `b` en la meta actual. `rewrite <- H` sustituye en sentido contrario.

`destruct x` Genera un sub-caso por cada constructor del tipo de `x`. Sobre un `bool` produce dos ramas: `true` y `false`. Sobre un `nat` produce `0` y `S n`.

Ejercicios

Los ejercicios siguen la línea del capítulo de *Software Foundations* (vol. 1, *Logical Foundations*). Con el objetivo de integrar esta introducción a los siguientes temas y recursos en los cuales se basa esta práctica, los ejercicios están agrupados por tema y ordenados de menor a mayor dificultad dentro de cada bloque. Los marcados con (*) son obligatorios con `Qed`.

Bloque 1 : Tipos, Check y Compute

Estos ejercicios no requieren `Proof`; basta ejecutarlos en el *scratchpad* y observar la salida.

1. Inspeccionando tipos

Escribe y ejecuta cada línea en el *scratchpad*. Anota el tipo que reporta Rocq para cada expresión y explica brevemente por qué tiene ese tipo.

```

Check true.
Check false.
Check negb.
Check (negb true).
Check nat.
Check (S (S 0)).
Check (1 + 1 = 2).
Check (forall n : nat, n = n).

```

Pregunta de reflexión: ¿Cuál es la diferencia entre `Check (1 + 1 = 2)` y `Compute (1 + 1)`?

2. Evaluación con `Compute`

Evalúa las siguientes expresiones con `Compute` y predice su resultado *antes* de ejecutarlas.

```

Compute negb (negb true).
Compute andb true false.
Compute orb false false.
Compute 3 + 4.
Compute 10 - 3.
Compute 10 - 15.
Compute 2 * 6.

```

Pregunta: ¿Qué ocurre con `10 - 15` en `nat`? ¿Por qué?

Bloque 2 : Definiciones y Fixpoint

1. Funciones booleanas básicas (*)

Define las siguientes funciones usando `Definition` y `match`. Verifica cada una con `Compute`.

```

(* nandb b1 b2 = NOT (b1 AND b2) *)
Definition nandb (b1 b2 : bool) : bool
  (* TU DEFINICION *).

(* xorb b1 b2 = verdadero ssi exactamente uno es true *)
Definition xorb (b1 b2 : bool) : bool
  (* TU DEFINICION *).

Example test_nandb1 : nandb true false = true. Proof. reflexivity. Qed.
Example test_nandb2 : nandb false false = true. Proof. reflexivity. Qed.
Example test_nandb3 : nandb true true = false. Proof. reflexivity. Qed.
Example test_xorb1 : xorb true false = true. Proof. reflexivity. Qed.
Example test_xorb2 : xorb true true = false. Proof. reflexivity. Qed.

```

2. Funciones sobre naturales

Define las siguientes funciones. No puedes usar operadores aritméticos predefinidos (+, *); solo `match` y llamadas recursivas.

```

(* par n = true ssi n es par *)
Fixpoint par (n : nat) : bool :=
  match n with
  | 0      => true
  | S 0    => false
  | S (S k) => par k
  end.

```

```

(* impar n = NOT (par n) -- una linea *)
Definition impar (n : nat) : bool
  (* TU DEFINICION *).

(* suma_nat n m sin usar + *)
Fixpoint suma_nat (n m : nat) : nat
  (* TU DEFINICION *).

(* mult_nat n m sin usar * ni + *)
Fixpoint mult_nat (n m : nat) : nat
  (* TU DEFINICION *).

```

Comprueba con:

```

Compute par 4.      (* true *)
Compute par 7.      (* false *)
Compute suma_nat 3 5. (* 8 *)
Compute mult_nat 3 4. (* 12 *)

```

3. Potencia (*)

Define `pot b e` que calcula b^e para naturales, sin usar el operador `^` de la biblioteca.

```

Fixpoint pot (b e : nat) : nat
  (* TU DEFINICION *).

Example test_pot1 : pot 2 0 = 1. Proof. reflexivity. Qed.
Example test_pot2 : pot 2 3 = 8. Proof. reflexivity. Qed.
Example test_pot3 : pot 3 2 = 9. Proof. reflexivity. Qed.

```

Pregunta: ¿Por qué Rocq acepta esta definición sin protestar por la terminación?

Bloque 3 : Teoremas simples con `reflexivity` y `simpl`

1. Demostraciones por evaluación (*)

Demuestra los siguientes resultados. Cada demostración debe usar solo `simpl` y/o `reflexivity`.

```

Theorem suma0_n : forall n : nat, 0 + n = n.
Proof.
  (* TU DEMOSTRACION *)
Qed.

Theorem negb_involutiva : negb (negb true) = true.
Proof.
  (* TU DEMOSTRACION *)
Qed.

Theorem andb_comm_ff : andb false false = andb false false.
Proof.
  (* TU DEMOSTRACION *)
Qed.

```

Nota: observa que `suma0_n` se demuestra sin inducción porque `0 + n` *reduce* a `n` por la definición de `+`. Prueba `n + 0 = n` con las mismas tácticas: ¿funciona? ¿Por qué?

Bloque 4 : Análisis por casos con destruct

1. Booleanos e identidades (*)

Usa destruct para demostrar:

```
Theorem negb_negb : forall b : bool, negb (negb b) = b.
Proof.
  intro b.
  destruct b.
  - (* caso b = true *) (* TU TACTICA *)
  - (* caso b = false *) (* TU TACTICA *)
Qed.

Theorem andb_false_r : forall b : bool, andb b false = false.
Proof.
  (* TU DEMOSTRACION *)
Qed.

Theorem orb_true_l : forall b : bool, orb true b = true.
Proof.
  (* TU DEMOSTRACION *)
Qed.
```

2. Conmutatividad de orb

Demuestra que orb es conmutativo. **Hint:** necesitarás destruct sobre *dos* variables booleanas.

```
Theorem orb_comm : forall b1 b2 : bool,
  orb b1 b2 = orb b2 b1.
Proof.
  intros b1 b2.
  destruct b1.
  - destruct b2.
    + (* b1=true, b2=true *) (* TU TACTICA *)
    + (* b1=true, b2=false *) (* TU TACTICA *)
  - destruct b2.
    + (* b1=false, b2=true *) (* TU TACTICA *)
    + (* b1=false, b2=false *) (* TU TACTICA *)
Qed.
```

Pregunta: ¿Cuántos sub-casos genera un destruct anidado sobre dos bool? ¿Y sobre dos nat? ¿Sería razonable hacerlo para naturales en general?

3. Casos sobre naturales

Demuestra:

```
(* Sugerencia: destruct n genera 0 y (S n') *)
Theorem par_S_impar : forall n : nat,
  par (S n) = negb (par n).
Proof.
  (* TU DEMOSTRACION *)
Admitted. (* cambia a Qed cuando lo completes *)
```

Pista: no necesitas inducción. Piensa primero cuántos casos tiene n y qué ocurre con la función par.

Bloque 5 : Reescritura con rewrite

1. Usando hipótesis (*)

Demuestra:

```

Theorem reescritura_simple :
  forall n m : nat, n = m -> n + 0 = m + 0.
Proof.
  intros n m H.
  rewrite H.
  (* TU TACTICA FINAL *)
Qed.

```

Ahora una variante con reescritura invertida:

```

Theorem reescritura_inv :
  forall n m : nat, m = n -> n + 0 = m + 0.
Proof.
  intros n m H.
  rewrite <- H.
  (* TU TACTICA FINAL *)
Qed.

```

2. Transitividad de la igualdad a mano

Sin usar lemas de la biblioteca, demuestra que la igualdad es transitiva usando solo `intros`, `rewrite` y `reflexivity`.

```

Theorem trans_eq_nat :
  forall a b c : nat, a = b -> b = c -> a = c.
Proof.
  (* TU DEMOSTRACION *)
Qed.

```

3. Congruencia y sustitución (*)

Demuestra:

```

(* Si n = m entonces sus sucesores son iguales *)
Theorem congr_S : forall n m : nat, n = m -> S n = S m.
Proof.
  (* TU DEMOSTRACION *)
Qed.

(* Si n = m, suma (S n) k = suma (S m) k *)
(* Usa la definicion de suma del Marco Teorico *)
Theorem congr_suma :
  forall n m k : nat, n = m -> suma (S n) k = suma (S m) k.
Proof.
  (* TU DEMOSTRACION *)
Qed.

```

Bloque 6 : Admitted y Abort (reflexión)

1. Explorando límites

El objetivo de este ejercicio es *entender* las herramientas de gestión de pruebas.

- a) Escribe el siguiente bloque tal cual y observa qué acepta Rocq:

```

Theorem ejemplo_admitted : forall n : nat, n + 0 = n.
Proof.
  Admitted.

```

b) Luego escribe un lema que *use* ejemplo_admitted:

```
Theorem usa_admitted : forall n : nat, n + 0 + 0 = n.
Proof.
  intro n.
  rewrite ejemplo_admitted.
  rewrite ejemplo_admitted.
  reflexivity.
Qed.
```

¿Compila? Anota la advertencia que da Rocq en la barra lateral.

c) Ahora prueba Abort:

```
Theorem intento_fallido : forall b : bool, b = true.
Proof.
  intro b.
  (* Nos atascamos aqui *)
  Abort.

(* Rocq no declara el teorema; podemos continuar normalmente *)
Check negb.
```

d) **Pregunta de reflexión:** ¿Cuándo conviene Admitted y cuándo Abort? ¿Qué riesgo implica dejar Admitted en un desarrollo grande?

Bloque 7 : Reto final (**)

Los siguientes resultados son demostrables *sin inducción* usando solo las tácticas permitidas. Si en algún punto sientes que “necesitas” inducción, es probable que estés atacando el problema de la forma incorrecta; reorganiza tu estrategia.

1. Propiedades de par e impar (**)

```
(* Demuestra los siguientes resultados. *)
(* Usa destruct y simpl generosamente. *)

Theorem par_0 : par 0 = true.
Proof. reflexivity. Qed.

Theorem impar_1 : impar 1 = true.
Proof. (* TU DEMOSTRACION *) Qed.

Theorem par_impar_complementarios :
  forall n : nat, impar n = negb (par n).
Proof.
  (* TU DEMOSTRACION *)
  (* Pista: que hace impar por definicion? *)
  Qed.
```

2. Mini-álgebra booleana (**)

Demuestra las leyes de De Morgan para booleanos:


```

(* NOT (A AND B) = (NOT A) OR (NOT B) *)
Theorem demorgan_and :
  forall a b : bool,
    negb (andb a b) = orb (negb a) (negb b).
Proof.
  intros a b.
  destruct a; destruct b; simpl; reflexivity.
  (* Comprime los 4 casos en una sola linea con punto y coma *)
Qed.

(* NOT (A OR B) = (NOT A) AND (NOT B) *)
Theorem demorgan_or :
  forall a b : bool,
    negb (orb a b) = andb (negb a) (negb b).
Proof.
  (* TU DEMOSTRACION -- intenta usar la misma estrategia *)
Qed.

```

Nota sobre el punto y coma: en Rocq, `tac1`; `tac2` aplica `tac2` a *todas* las metas generadas por `tac1`. Esto permite comprimir casos simétricos. Observa cómo `demorgan_and` usa este patrón y replica la idea.

3. Función de comparación (**)

Define una función que compare dos naturales:

```

(* Resultado: Eq | Lt | Gt *)
Inductive comparacion : Type :=
| Eq : comparacion
| Lt : comparacion
| Gt : comparacion.

Fixpoint compara (n m : nat) : comparacion :=
  match n, m with
  | 0, 0   => Eq
  | 0, S _ => Lt
  | S _, 0 => Gt
  | S n', S m' => compara n' m'
  end.

```

Demuestra:

```

Theorem compara_refl : forall n : nat, compara n n = Eq.
Proof.
  intro n.
  (* Pista: destruct n genera 0 y (S n'), pero en el caso
     recursivo necesitas algo mas.... *)
  Admitted.

Theorem compara_0_n : forall n : nat, compara 0 (S n) = Lt.
Proof.
  (* TU DEMOSTRACION *)
Qed.

```

Reflexión: `compara_refl` es difícil de demostrar sin inducción. Descríbelo en tu reporte: ¿qué táctica falta? ¿Qué ocurre si escribes `Admitted`?

Entrega

1. Entrega un único archivo `.v` con todos los ejercicios.

2. Cada ejercicio debe incluir los `Example/Compute` de verificación donde se indiquen.
3. Los ejercicios marcados (*) deben terminar en `Qed`. Los demás pueden usar `Admitted`, pero debes indicarlo *con un comentario* que explique qué te faltó.
4. No se aceptan archivos que **no compilen** en jsCoq.
5. Deben entregar un archivo `README.md` con las respuestas a preguntas y demás indicaciones. Este debe ser necesariamente largo, es decir, eviten duplicidad en sus respuestas. Las reflexiones deben ser breves y responder claramente a la cuestión.

Consideraciones

1. Las únicas soluciones iguales admisibles son las resueltas en conjunto durante el laboratorio.
2. Cada día de retraso se penalizará con un punto sobre la calificación.
3. **No** se permite el uso de tácticas distintas a las listadas en las instrucciones.
4. No se modificará la definición de las funciones ni los tipos ya dados.
5. No se recibirán prácticas que no compilen en jsCoq. Si no resuelven alguna demostración, déjenla con `Admitted` e indíquenlo con un comentario.
6. Las participaciones en el laboratorio se aplican individualmente.

Referencias

- [1] Software foundations. Accessed: Feb. 11, 2026.
- [2] The Rocq Development Team. Rocq standard library. Accessed: May 18, 2026.